

Respuestas al cuestionario técnico (Sección 3)

1. CI/CD Auto-hospedado vs. Servicio Gestionado: Desde una perspectiva de arquitectura y operación, ¿qué ventajas y responsabilidades adicionales introduce mantener Jenkins, Nexus y el relay de webhooks bajo administración propia, en comparación con utilizar una plataforma CI/CD gestionada como GitHub Actions? Considere aspectos como control, disponibilidad, mantenimiento, seguridad y costos operativos.

tener jenkins, nexus y el relay de webhooks propios da mas control sobre como funciona todo el proceso de ci/cd. uno puede decidir como configurar los pipelines, los usuarios, los permisos, los repositorios y la red. el problema es que tambien toca encargarse del mantenimiento, las actualizaciones, la seguridad, los respaldos y solucionar cualquier falla que tenga la infraestructura. con github actions muchas de esas tareas ya vienen administradas por github, entonces es mas facil de usar, pero se tiene menos control directo sobre la infraestructura.

2. Trazabilidad y Gobernanza de Artefactos: ¿Por qué la separación entre el código fuente, el proceso de construcción y el repositorio de artefactos es importante para garantizar la trazabilidad y reproducibilidad de una entrega? Explique cómo la relación entre commit, número de build, imagen Docker y artefacto Maven contribuye a identificar exactamente qué versión fue construida y desplegada.

1. separar el codigo fuente, el proceso de construccion y los artefactos ayuda a tener una mejor trazabilidad. el codigo queda guardado en git, jenkins registra cada build que se ejecuta y nexus guarda los artefactos generados. si el numero del build y el commit tambien se usan en la version maven o en la etiqueta de la imagen docker, se puede saber exactamente de que codigo salio cada version. esto ayuda mucho cuando hay errores, porque se puede identificar que version fue construida, cual fue desplegada y volver a una version anterior si es necesario.

Bitácora de prompts IAG.

Problemas con maven, olvide que no tenía maven instalado

```
Hello Minecraft! Launcher Stable
HMCL.HMCL.Stable.Maven.TencentCloudMirror 3.6.18      winget
PS C:\Windows\system32>
```

Perfecto. `winget` no tiene Apache Maven disponible en tu catálogo, pero **Java sí está bien instalado**. La ruta más directa ahora es instalar Maven manualmente; Apache indica que en Windows basta con descargar el ZIP, extraerlo y agregar su carpeta `bin` al `PATH`.  Apache Maven +1

Actualmente Apache publica **Maven 3.10.0** como la versión recomendada, compatible con tu JDK 25.

 Apache Maven

Haz esto:

1. Descarga el ZIP binario de Maven 3.10.0 desde la página oficial: [Descargar Apache Maven](#) »

Busca:

</> Texto sin formato

🔍 📄

apache-maven-3.10.0-bin.zip

2. Descomprímelo, por ejemplo en:

</> Texto sin formato

🔍 📄

C:\Program Files\Apache\maven

De forma que quede algo como:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\System32> mvn -version
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: C:\Apache\maven\apache-maven-3.10.0
Java version: 25.0.3, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot
Default locale: en_US, platform encoding: UTF-8, time zone: America/Bogota
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
PS C:\Windows\System32>
```

Problemas con flujo a seguir para saber si mis correcciones fueron bien hechas

ya esta el codigo creo que ya corriji todo cuales serian los comandos a probar antes de subir todo?

Procesó durante 1m 51s

Sí. Antes del `git push`, yo haría este **preflight en este orden**. Está adaptado a este taller: Nexus 9181/9180, Jenkins 9182, backend 8080, frontend 3000, red `cicd_network` y ejecución manual desde Git Bash. No necesitas scripts auxiliares.  Texto pegado

1. Desde la raíz, revisar Git

```
</> Bash
git status
git diff --check
git diff
```

`git diff --check` no debería imprimir nada.

Comprueba que no estás subiendo secretos ni basura:

```
</> Bash
git ls-files | grep -E '^(^|/)\.env$'
git ls-files | grep -E '(target/|dist/|node_modules/|\.idea/|\.iml$)'
```

Ambos deberían quedar **sin salida**.

Busca cosas olvidadas:

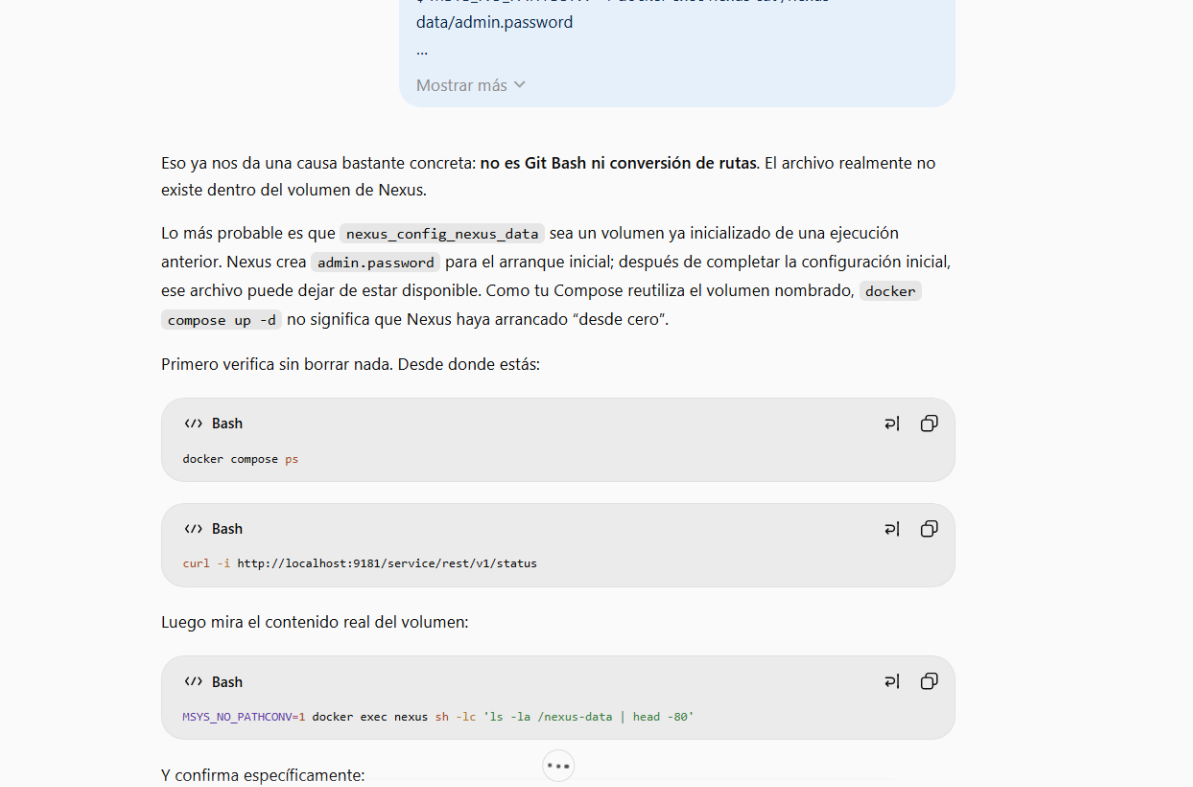


+ Pregúntale a ChatGPT

Alta ▾



problema con la contraseña de nexus



data/admin.password
...
Mostrar más ▾

Eso ya nos da una causa bastante concreta: **no es Git Bash ni conversión de rutas**. El archivo realmente no existe dentro del volumen de Nexus.

Lo más probable es que `nexus_config_nexus_data` sea un volumen ya inicializado de una ejecución anterior. Nexus crea `admin.password` para el arranque inicial; después de completar la configuración inicial, ese archivo puede dejar de estar disponible. Como tu Compose reutiliza el volumen nombrado, `docker compose up -d` no significa que Nexus haya arrancado "desde cero".

Primero verifica sin borrar nada. Desde donde estás:

```
</> Bash
docker compose ps
```

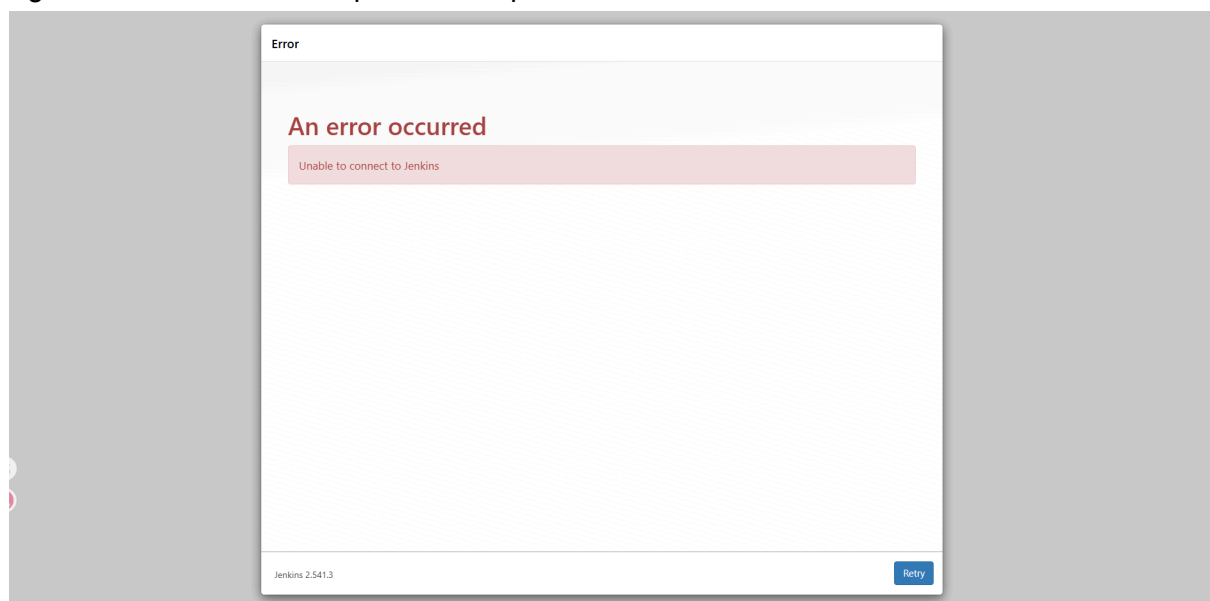
```
</> Bash
curl -i http://localhost:9181/service/rest/v1/status
```

Luego mira el contenido real del volumen:

```
</> Bash
MSYS_NO_PATHCONV=1 docker exec nexus sh -lc 'ls -la /nexus-data | head -80'
```

Y confirma específicamente:

Jenkins no me funciona la el contenedor no me dejaba usar el login y hasta ahí llegue sin poder hacer esa parte
logre entrar de una forma que conocia pero no funciona



se logro luego de recordar los videos del monitor.